

```

/* =====
CURSORS IN ORACLE PL/SQL

Topics:
1. Implicit Cursor
   - SQL%FOUND => found =true, not found = false
   - SQL%NOTFOUND =>not found = true, found = false
   - SQL%ROWCOUNT => return number of rows
   - SQL%ISOPEN => true otherwise false

2. Explicit Cursor
   - Cursor Declaration
   - OPEN
   - FETCH
   - CLOSE

3. Cursor FOR LOOP

4. Parameterized Cursor

Labs:
- Employee Salary Report
- Customer Statement Generator
- Telecom Subscriber Processing
===== */

SET SERVEROUTPUT ON SIZE UNLIMITED;

/* =====
1. DROP OLD TABLES
===== */

BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE employees CASCADE CONSTRAINTS';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;
/

BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE customers CASCADE CONSTRAINTS';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;
/

BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE telecom_subscribers CASCADE
CONSTRAINTS';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;
/

```

```

/* =====
2. CREATE TABLES
===== */

CREATE TABLE employees
(
    employee_id    NUMBER PRIMARY KEY,
    employee_name  VARCHAR2(100),
    department     VARCHAR2(50),
    salary         NUMBER,
    status         VARCHAR2(20)
);

CREATE TABLE customers
(
    customer_id    NUMBER PRIMARY KEY,
    customer_name  VARCHAR2(100),
    city           VARCHAR2(50),
    balance        NUMBER,
    status         VARCHAR2(20)
);

CREATE TABLE telecom_subscribers
(
    subscriber_id  NUMBER PRIMARY KEY,
    msisdn        VARCHAR2(20),
    customer_name  VARCHAR2(100),
    plan_name     VARCHAR2(50),
    balance        NUMBER,
    status         VARCHAR2(20)
);

/* =====
3. INSERT SAMPLE DATA
===== */

INSERT INTO employees VALUES (1, 'Aung Aung', 'IT', 900000, 'ACTIVE');
INSERT INTO employees VALUES (2, 'Mg Mg', 'HR', 600000, 'ACTIVE');
INSERT INTO employees VALUES (3, 'Su Su', 'Finance', 1200000, 'ACTIVE');
INSERT INTO employees VALUES (4, 'Hla Hla', 'IT', 750000, 'ACTIVE');
INSERT INTO employees VALUES (5, 'Kyaw Kyaw', 'Sales', 500000,
'INACTIVE');

INSERT INTO customers VALUES (1, 'ABC Trading', 'Yangon', 150000,
'ACTIVE');
INSERT INTO customers VALUES (2, 'Global Tech', 'Mandalay', 300000,
'ACTIVE');
INSERT INTO customers VALUES (3, 'Myanmar Retail', 'Yangon', 50000,
'INACTIVE');

INSERT INTO telecom_subscribers VALUES (1, '0911111111', 'Aung Aung',
'PREPAID', 5000, 'ACTIVE');
INSERT INTO telecom_subscribers VALUES (2, '0922222222', 'Mg Mg',
'POSTPAID', 25000, 'ACTIVE');

```

```

INSERT INTO telecom_subscribers VALUES (3, '093333333', 'Su Su',
'PREPAID', 1000, 'ACTIVE');
INSERT INTO telecom_subscribers VALUES (4, '094444444', 'Hla Hla',
'PREPAID', 0, 'SUSPENDED');

COMMIT;

/* =====
PART A: IMPLICIT CURSOR
Oracle automatically creates an implicit cursor for:
INSERT, UPDATE, DELETE, SELECT INTO
===== */

/* =====
4. SQL%FOUND Example

SQL%FOUND returns TRUE if DML affects at least one row.
===== */

BEGIN
    UPDATE employees
    SET salary = salary + 50000
    WHERE employee_id = 9;

    IF SQL%FOUND THEN
        DBMS_OUTPUT.PUT_LINE('SQL%FOUND: Employee salary updated.');

```

```

SQL> BEGIN
    UPDATE employees
    SET salary = salary + 50000
    WHERE employee_id = 9;

    IF SQL%FOUND THEN
        DBMS_OUTPUT.PUT_LINE('SQL%FOUND: Employee salary updated.');

```

2	3	4	5	6	7	8	9	10	11	12	13	14	Not found
---	---	---	---	---	---	---	---	----	----	----	----	----	-----------

```

    ELSE
        DBMS_OUTPUT.PUT_LINE('Not found');
    END IF;

    COMMIT;
END;
/

PL/SQL procedure successfully completed.
```

```

SQL> DECLARE
    v_sal employees.salary%TYPE;
BEGIN
    SELECT salary into v_sal FROM employees
    WHERE employee_id = 9;

    EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No Employee :Error in employees.');
```

2	3	4	5	6	7	8	9	10	11	12	13	No Employee :Error in employees.
---	---	---	---	---	---	---	---	----	----	----	----	----------------------------------

```

    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Unexpected Error: ' || SQLERRM);
END;
/

PL/SQL procedure successfully completed.
```

```

/* =====
5. SQL%NOTFOUND Example

SQL%NOTFOUND returns TRUE if DML affects no row.
===== */
```

```

SQL> SQL> BEGIN
    UPDATE employees
    SET salary = salary + 50000
    WHERE employee_id = 999;

    IF SQL%NOTFOUND THEN
        DBMS_OUTPUT.PUT_LINE('SQL%NOTFOUND: No employee found.');
```

2	3	4	5	6	7	8	9	10	11	12	SQL%NOTFOUND: No employee found.
---	---	---	---	---	---	---	---	----	----	----	----------------------------------

```

    END IF;

    COMMIT;
END;
/

PL/SQL procedure successfully completed.
```

```

/* =====
6. SQL%ROWCOUNT Example

SQL%ROWCOUNT returns number of affected rows.
===== */

```

```

SQL> BEGIN
  UPDATE employees
  SET salary = salary + 100000
  WHERE department = 'IT';

  DBMS_OUTPUT.PUT_LINE('SQL%ROWCOUNT: Updated rows = ' || SQL%ROWCOUNT);

  COMMIT;
END;
/

  2      3      4      5      6      7      8      9     10 SQL%ROWCOUNT: Updated rows = 2
PL/SQL procedure successfully completed.
SQL> SQL> Select employee_id, salary from employees where department='IT';

EMPLOYEE_ID      SALARY
-----
          1      1000000
          4       850000

```

```

/* =====
7. SQL%ISOPEN Example

For implicit cursor, SQL%ISOPEN is always FALSE.
Oracle automatically opens and closes implicit cursor.
===== */

```

```

SQL> BEGIN
  UPDATE employees
  SET salary = salary
  WHERE department = 'HR';

  IF SQL%ISOPEN THEN
    DBMS_OUTPUT.PUT_LINE('Implicit cursor is open.');
```

```

  ELSE
    DBMS_OUTPUT.PUT_LINE('SQL%ISOPEN: Implicit cursor is always closed.');
```

```

  END IF;

  COMMIT;
END;
/

  2      3      4      5      6      7      8      9     10     11     12     13     14 SQL%ISOPEN: Implicit
cursor is always closed.

PL/SQL procedure successfully completed.

```

```

/* =====
PART B: EXPLICIT CURSOR
Explicit cursor is manually declared, opened, fetched, closed.
===== */

/* =====
8. Explicit Cursor Example
===== */

```

```

SQL> DECLARE
  CURSOR cur_emp IS
    SELECT employee_id, employee_name, department, salary
    FROM employees
    WHERE status = 'ACTIVE';
  v_employee_id   employees.employee_id%TYPE;
  v_employee_name employees.employee_name%TYPE;
  v_department    employees.department%TYPE;
  v_salary        employees.salary%TYPE;
BEGIN
  OPEN cur_emp;

  LOOP
    FETCH cur_emp
    INTO v_employee_id, v_employee_name, v_department, v_salary;
    EXIT WHEN cur_emp%NOTFO 2 UND;

    DBMS_OUTPUT.PUT_LINE(
      v_employee_id || ' - ' ||
      v_employee_name || ' - ' ||
      v_department || ' - ' ||
      v_salary
    );
  END LOOP;
  CLOSE cur_emp;
END;
/

```

	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
20	21	22	23	24	25	26	27	1	-	Aung	Aung	-	IT	-	1000000		
2 - Mg Mg - HR - 600000																	
3 - Su Su - Finance - 1200000																	
4 - Hla Hla - IT - 850000																	

```

PL/SQL procedure successfully completed.

```

```

SQL> SELECT employee_id, employee_name, department, salary
        FROM employees
        WHERE status = 'ACTIVE';

```

EMPLOYEE_ID	EMPLOYEE_NAME	DEPARTMENT	SALARY
2	Aung Aung	IT	1000000
3	Mg Mg	HR	600000
3	Su Su	Finance	1200000
4	Hla Hla	IT	850000

```

/* =====
9. Explicit Cursor with Cursor Attributes
===== */

```

```

DECLARE
    CURSOR cur_customer IS
        SELECT customer_id, customer_name, balance
        FROM customers
        WHERE status = 'ACTIVE';

    v_customer_id    customers.customer_id%TYPE;
    v_customer_name  customers.customer_name%TYPE;
    v_balance        customers.balance%TYPE;
BEGIN
    OPEN cur_customer;

    LOOP
        FETCH cur_customer

```

```

        INTO v_customer_id, v_customer_name, v_balance;

        EXIT WHEN cur_customer%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            'Customer: ' || v_customer_name ||
            ', Balance: ' || v_balance
        );

        DBMS_OUTPUT.PUT_LINE(
            'Fetched Row Count: ' || cur_customer%ROWCOUNT
        );
    END LOOP;

    IF cur_customer%ISOPEN THEN
        DBMS_OUTPUT.PUT_LINE('Cursor is still open.');
```

```

    END IF;
```

```

    CLOSE cur_customer;
```

```

END;
```

```

/
```

```

Customer: ABC Trading, Balance: 150000
Fetched Row Count: 1
Customer: Global Tech, Balance: 300000
Fetched Row Count: 2
Cursor is still open.

PL/SQL procedure successfully completed.
```



```

/* =====
PART C: CURSOR FOR LOOP
Cursor FOR LOOP automatically:
- Opens cursor
- Fetches rows
- Closes cursor
===== */

```

```

SQL> SQL> BEGIN
FOR emp_rec IN
(
  SELECT employee_id, employee_name, department, salary
  FROM employees
  WHERE status = 'ACTIVE'
)
LOOP
  DBMS_OUTPUT.PUT_LINE(
    emp_rec.employee_id || ' - ' ||
    emp_rec.employee_name || ' - ' ||
    emp_rec.department || ' - ' ||
    emp_rec.salary
  );
END LOOP;
END;
/

```

	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
1 -	Aung	Aung	-	IT	-	1000000										
2 -	Mg	Mg	-	HR	-	600000										
3 -	Su	Su	-	Finance	-	1200000										
4 -	Hla	Hla	-	IT	-	850000										

```

PL/SQL procedure successfully completed.

```

```

/* =====
PART D: PARAMETERIZED CURSOR
Parameterized cursor accepts input values.
===== */

```

```

SQL> DECLARE
CURSOR cur_emp_by_dept
(
    p_department employees.department%TYPE
)
IS
    SELECT employee_id, employee_name, salary
    FROM employees
    WHERE department = p_department
    AND status = 'ACTIVE';

BEGIN
    DBMS_OUTPUT.PUT_LINE('Employees from IT Department');

    FOR emp_rec IN cur_emp_by_dept('IT')
    LOOP
        DBMS_OUTPUT.PUT_LINE(
            emp_rec.employee_id || ' - ' ||
            emp_rec.employee_name || ' - ' ||
            emp_rec.salary
        );
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Employees from Finance Department');

    FOR emp_rec IN cur_emp_by_dept('Finance')
    LOOP
        DBMS_OUTPUT.PUT_LINE(
            emp_rec.employee_id || ' - ' ||
            emp_rec.employee_name || ' - ' ||
            emp_rec.salary
        );
    END LOOP;
END;
/

```

3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	1
9	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35

```

Employees from IT Department
1 - Aung Aung - 1000000
4 - Hla Hla - 850000
Employees from Finance Department
3 - Su Su - 1200000

PL/SQL procedure successfully completed.

```

```

/* =====
LAB 1: EMPLOYEE SALARY REPORT
Requirement:
- Show active employees
- Calculate annual salary
- Classify salary level
===== */

DECLARE
    CURSOR cur_salary_report IS
        SELECT employee_id, employee_name, department, salary
        FROM employees
        WHERE status = 'ACTIVE'
        ORDER BY department, employee_id;

    v_annual_salary NUMBER;
    v_salary_level VARCHAR2(30);
BEGIN
    DBMS_OUTPUT.PUT_LINE('===== EMPLOYEE SALARY REPORT =====');

    FOR emp_rec IN cur_salary_report
    LOOP
        v_annual_salary := emp_rec.salary * 12;

        IF emp_rec.salary >= 1000000 THEN
            v_salary_level := 'HIGH SALARY';
        ELSIF emp_rec.salary >= 700000 THEN
            v_salary_level := 'MEDIUM SALARY';
        ELSE
            v_salary_level := 'LOW SALARY';
        END IF;

        DBMS_OUTPUT.PUT_LINE(
            'ID: ' || emp_rec.employee_id ||
            ', Name: ' || emp_rec.employee_name ||
            ', Dept: ' || emp_rec.department ||
            ', Monthly Salary: ' || emp_rec.salary ||
            ', Annual Salary: ' || v_annual_salary ||
            ', Level: ' || v_salary_level
        );
    END LOOP;
END;
/

```

```

===== EMPLOYEE SALARY REPORT =====
ID: 3, Name: Su Su, Dept: Finance, Monthly Salary: 1200000, Annual Salary:
14400000, Level: HIGH SALARY
ID: 2, Name: Mg Mg, Dept: HR, Monthly Salary: 600000, Annual Salary: 7200000,
Level: LOW SALARY
ID: 1, Name: Aung Aung, Dept: IT, Monthly Salary: 1000000, Annual Salary:
12000000, Level: HIGH SALARY
ID: 4, Name: Hla Hla, Dept: IT, Monthly Salary: 850000, Annual Salary: 10200000,
Level: MEDIUM SALARY

```

PL/SQL procedure successfully completed.

```

/* =====
LAB 2: CUSTOMER STATEMENT GENERATOR
Requirement:
- Generate statement for active customers
- Show customer balance
- Classify customer account status
===== */

DECLARE
    CURSOR cur_customer_statement IS
        SELECT customer_id, customer_name, city, balance
        FROM customers
        WHERE status = 'ACTIVE'
        ORDER BY customer_id;

    v_balance_status VARCHAR2(50);
BEGIN
    DBMS_OUTPUT.PUT_LINE('===== CUSTOMER STATEMENT =====');

    FOR cust_rec IN cur_customer_statement
    LOOP
        IF cust_rec.balance >= 200000 THEN
            v_balance_status := 'GOOD BALANCE';
        ELSIF cust_rec.balance >= 100000 THEN
            v_balance_status := 'NORMAL BALANCE';
        ELSE
            v_balance_status := 'LOW BALANCE';
        END IF;

        DBMS_OUTPUT.PUT_LINE(
            'Customer ID: ' || cust_rec.customer_id ||
            ', Name: ' || cust_rec.customer_name ||
            ', City: ' || cust_rec.city ||
            ', Balance: ' || cust_rec.balance ||
            ', Status: ' || v_balance_status
        );
    END LOOP;
END;
/

```

```

===== CUSTOMER STATEMENT =====
Customer ID: 1, Name: ABC Trading, City: Yangon, Balance: 150000, Status: NORMAL
BALANCE
Customer ID: 2, Name: Global Tech, City: Mandalay, Balance: 300000, Status: GOOD
BALANCE

PL/SQL procedure successfully completed.

```

```

/* =====
LAB 3: TELECOM SUBSCRIBER PROCESSING
Requirement:
- Process active telecom subscribers
- Check prepaid/postpaid plan
- Warn prepaid users with low balance
===== */

DECLARE
    CURSOR cur_subscriber
    (
        p_status telecom_subscribers.status%TYPE
    )
    IS
        SELECT subscriber_id, msisdn, customer_name, plan_name, balance
        FROM telecom_subscribers
        WHERE status = p_status
        ORDER BY subscriber_id;

    v_message VARCHAR2(200);
BEGIN
    DBMS_OUTPUT.PUT_LINE('==== TELECOM SUBSCRIBER PROCESSING ====');

    FOR sub_rec IN cur_subscriber('ACTIVE')
    LOOP
        IF sub_rec.plan_name = 'PREPAID' AND sub_rec.balance <= 2000 THEN
            v_message := 'LOW BALANCE WARNING';
        ELSIF sub_rec.plan_name = 'PREPAID' THEN
            v_message := 'PREPAID ACCOUNT NORMAL';
        ELSIF sub_rec.plan_name = 'POSTPAID' THEN
            v_message := 'POSTPAID BILLING ACCOUNT';
        ELSE
            v_message := 'UNKNOWN PLAN';
        END IF;

        DBMS_OUTPUT.PUT_LINE(
            'Subscriber ID: ' || sub_rec.subscriber_id ||
            ', MSISDN: ' || sub_rec.msisdn ||
            ', Name: ' || sub_rec.customer_name ||
            ', Plan: ' || sub_rec.plan_name ||
            ', Balance: ' || sub_rec.balance ||
            ', Message: ' || v_message
        );
    END LOOP;
END;
/

```

```

==== TELECOM SUBSCRIBER PROCESSING ====
Subscriber ID: 1, MSISDN: 091111111, Name: Aung Aung, Plan: PREPAID, Balance:
5000, Message: PREPAID ACCOUNT NORMAL
Subscriber ID: 2, MSISDN: 092222222, Name: Mg Mg, Plan: POSTPAID, Balance:
25000, Message: POSTPAID BILLING ACCOUNT
Subscriber ID: 3, MSISDN: 093333333, Name: Su Su, Plan: PREPAID, Balance: 1000,
Message: LOW BALANCE WARNING

```

```
PL/SQL procedure successfully completed.
```

```
/* =====  
FINAL CHECKING  
===== */
```

```
SQL> SELECT * FROM employees ORDER BY employee_id;
```

```
EMPLOYEE_ID
```

```
EMPLOYEE_NAME
```

```
DEPARTMENT
```

```
SALARY
```

```
STATUS
```

```
1
```

```
Aung Aung
```

```
IT
```

```
1000000
```

```
ACTIVE
```

```
EMPLOYEE_ID
```

```
EMPLOYEE_NAME
```

```
DEPARTMENT
```

```
SALARY
```

```
STATUS
```

```
2
```

```
Mg Mg
```

```
HR
```

```
600000
```

```
ACTIVE
```

```
EMPLOYEE_ID
-----
EMPLOYEE_NAME
-----
DEPARTMENT                                SALARY
-----
STATUS
-----
          3
Su Su
Finance                                1200000
ACTIVE

EMPLOYEE_ID
-----
EMPLOYEE_NAME
-----
DEPARTMENT                                SALARY
-----
STATUS
-----
          4
Hla Hla
IT                                    850000
ACTIVE

EMPLOYEE_ID
-----
EMPLOYEE_NAME
-----
DEPARTMENT                                SALARY
-----
STATUS
-----
          5
Kyaw Kyaw
Sales                                500000
INACTIVE
```

```
SQL> SELECT * FROM customers ORDER BY customer_id;
```

```
CUSTOMER_ID
```

```
CUSTOMER_NAME
```

```
CITY
```

```
BALANCE
```

```
STATUS
```

```
1
```

```
ABC Trading
```

```
Yangon
```

```
150000
```

```
ACTIVE
```

```
CUSTOMER_ID
```

```
CUSTOMER_NAME
```

```
CITY
```

```
BALANCE
```

```
STATUS
```

```
2
```

```
Global Tech
```

```
Mandalay
```

```
300000
```

```
ACTIVE
```

```
CUSTOMER_ID
```

```
CUSTOMER_NAME
```

```
CITY
```

```
BALANCE
```

```
STATUS
```

```
3
```

```
Myanmar Retail
```

```
Yangon
```

```
50000
```

```
INACTIVE
```



```
SQL> SELECT * FROM telecom_subscribers ORDER BY subscriber_id;
```

```
SUBSCRIBER_ID MSISDN
```

```
CUSTOMER_NAME
```

```
PLAN_NAME
```

```
BALANCE
```

```
STATUS
```

```
1 0911111111
```

```
Aung Aung
```

```
PREPAID
```

```
5000
```

```
ACTIVE
```

```
SUBSCRIBER_ID MSISDN
```

```
CUSTOMER_NAME
```

```
PLAN_NAME
```

```
BALANCE
```

```
STATUS
```

```
2 0922222222
```

```
Mg Mg
```

```
POSTPAID
```

```
25000
```

```
ACTIVE
```

```
SUBSCRIBER_ID MSISDN
```

```
CUSTOMER_NAME
```

```
PLAN_NAME
```

```
BALANCE
```

```
STATUS
```

```
3 0933333333
```

```
Su Su
```

```
PREPAID
```

```
1000
```

```
ACTIVE
```

```
SUBSCRIBER_ID MSISDN
```

```
CUSTOMER_NAME
```

```
PLAN_NAME
```

```
BALANCE
```

```
STATUS
```

```
4 0944444444
```

```
Hla Hla
```

```
PREPAID
```

```
0
```

```
SUSPENDED
```